

Clase 065 — Implementaciones seguras y errores criptográficos comunes

Parte: 2 — **Criptografía aplicada** · Fuente: *Cryptography Engineering* (Ferguson/Schneier/Kohno) y *Real-World Cryptography* (Wong) 🕒 Duración estimada: **120 min** · Nivel: **Avanzado**

🎯 Objetivo

Cerrar la parte con la lección que unifica todo lo anterior: la criptografía se rompe en la implementación, no en las matemáticas. El alumno recopilará el catálogo de errores criptográficos comunes (los del OWASP A02 "Cryptographic Failures"), aprenderá a auditarlos y evitarlos, y consolidará las reglas de oro: usa librerías auditadas, AEAD por defecto, nonces únicos, aleatoriedad segura, comparación en tiempo constante y no inventes tu propia cripto.

📊 Resultados de aprendizaje

Al finalizar, el alumno podrá:

- 1. **Enumerar** los fallos criptográficos más comunes y su causa raíz.
- 2. **Auditar** un fragmento de código en busca de anti-patrones cripto.
- 3. **Aplicar** las reglas de oro de implementación segura.
- 4. **Elegir** las primitivas por defecto correctas para cada necesidad.
- 5. **Usar** herramientas de detección (linters cripto, escáneres de secretos).

📖 Temas

#	Tema	Por qué importa
1	OWASP A02 Cryptographic Failures	Marco de referencia
2	Primitivas obsoletas (DES, MD5, ECB, RC4)	Erradicarlas
3	Nonces/IV mal gestionados	Fuente recurrente de brechas
4	Aleatoriedad y claves hardcodeadas	Fallos silenciosos
5	Falta de autenticación (sin AEAD/MAC)	Manipulación
6	Comparaciones no constantes	Timing
7	Reglas de oro y "criptoagilidad"	Diseño robusto y migrable

📖 Definiciones y características

- **Cryptographic Failures (OWASP A02):** categoría que agrupa el uso de cripto débil, mal configurada o ausente que expone datos sensibles.

- **Criptoagilidad:** capacidad de un sistema para cambiar algoritmos y claves sin rediseñarse; clave para la migración PQC.
- **Anti-patrón cripto:** práctica insegura recurrente (ECB, IV fijo, clave en código, hash rápido para contraseñas, comparación con `==`).
- **Defaults seguros:** elegir de fábrica AEAD, KDFs lentas y curvas modernas para que el camino fácil sea el correcto.
- **Superficie de implementación:** todo el código que rodea la primitiva (padding, parsing, manejo de errores) donde suelen estar los bugs.
- **Fail closed:** ante error, denegar sin exponer datos ni detalles.
- **Gestión del ciclo de vida de claves:** generación, distribución, rotación y destrucción seguras.

Herramientas y preparación

```
pip install cryptography bandit
which gitleaks semgrep 2>/dev/null || echo "opcional: escáneres estáticos"
```

Auditoría sobre código propio o autorizado.

Laboratorio guiado

1. **Auditoría de código guiada.** Toma este fragmento vulnerable y detecta todos los fallos:

```
python import hashlib, random KEY = b"1234567890123456" # clave hardcoded def cifrar(m):
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes iv = b"\x00" * 16
# IV fijo c = Cipher(algorithms.AES(KEY), modes.ECB()) # ECB, sin auth return
c.encryptor().update(m) def token(): return str(random.random()) # PRNG no seguro def check(a,
b): return a == b # comparación no constante def pwd_hash(p): return
hashlib.md5(p).hexdigest() # MD5 para contraseña
```

Identifica: clave en código, ECB, sin autenticación, IV fijo, PRNG inseguro, comparación no constante y MD5 para contraseñas.

2. **Reescríbelo de forma segura:** clave desde KMS/Vault o variable de entorno, AES-GCM (AEAD) con nonce del CSPRNG, `secrets` para tokens, `hmac.compare_digest`, Argon2id para contraseñas.
3. **Escáneres estáticos.** Corre `bandit` sobre el archivo y `gitleaks` / `semgrep` para detectar la clave hardcoded; compara hallazgos con tu revisión manual.
4. **Checklist de cryptoagilidad.** Verifica que tu diseño permite cambiar algoritmo/clave por configuración y versiona el formato de los mensajes cifrados.
5. **Revisión de dependencias.** Comprueba que usas librerías mantenidas (`cryptography` , `libsodium`) y no implementaciones caseras de primitivas.

Ejercicios

1. Lista siete anti-patrones cripto y su corrección.
2. Reescribe el fragmento vulnerable del laboratorio de forma segura.
3. Ejecuta `bandit` y explica cada advertencia relevante.

4. Diseña un formato de mensaje cifrado con versión para criptoagilidad.
5. Audita una configuración TLS y una de almacenamiento de contraseñas juntas.
6. Redacta una guía interna de "reglas de oro cripto" para tu equipo.

Reto verificable

Toma un módulo con al menos cinco fallos criptográficos (el del laboratorio u otro que construyas) y entrégalo corregido: AEAD, nonces del CSPRNG, claves fuera del código, Argon2id para contraseñas y comparaciones en tiempo constante, con versión de formato para criptoagilidad. **Criterio de aceptación:** `bandit` no reporta fallos cripto de severidad media/alta en el módulo corregido, los tests de cifrado/descifrado y de autenticación pasan, y ninguna clave o secreto aparece en el código.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Clave/secreto en el repositorio	Muévelo a KMS/Vault/variables; escanea el histórico
ECB o cifrado sin autenticar	Usa AEAD (GCM/ChaCha20-Poly1305)
IV/nonce fijo o reutilizado	Genera con CSPRNG; garantiza unicidad
MD5/SHA-1/DES/RC4 en uso	Sustituye por SHA-256/3, AES-GCM, Argon2id
Comparaciones con <code>==</code> de secretos	Usa <code>compare_digest</code> /verificadores de la librería
"Cripto propia" sin auditar	Usa librerías estándar bien mantenidas

? Preguntas frecuentes

? **¿Cuál es la regla más importante?** No inventes tu propia cripto: usa primitivas y librerías auditadas con defaults seguros (AEAD, KDFs lentas, curvas modernas).

? **¿Cómo detecto estos fallos a escala?** Combina revisión manual con escáneres (`bandit`, `semgrep`, `gitLeaks`) integrados en CI, y auditorías periódicas de TLS y almacenamiento.

? **¿Qué es la criptoagilidad y por qué me importa ahora?** Poder cambiar algoritmos y claves sin rediseñar; será imprescindible en la migración post-cuántica y ante cualquier primitiva que se debilite.

Referencias

- OWASP Top 10 A02:2021 Cryptographic Failures — https://owasp.org/Top10/A02_2021-Cryptographic_Failures/
- Ferguson, Schneier, Kohno, *Cryptography Engineering* (todo el libro).
- Wong, *Real-World Cryptography*, cap. 8, 13 y 16.
- OWASP Cryptographic Storage Cheat Sheet — https://cheatsheetseries.owasp.org/cheatsheets/Cryptographic_Storage_Cheat_Sheet.html

Siguiente clase